

CSE 220 Lab Test Complete Preparation: Fourier Series + Continuous Fourier Transform

Purpose: Night-before / lab-test-ready documentation for the exact style of CSE 220 FS/CFT coding questions seen in the supplied offline assignments and previous online questions.

Transform convention used throughout this document

\$\$

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi f t} dt,$$

$\Leftrightarrow$

$$x(t) = \int_{-\infty}^{\infty} X(f) e^{j2\pi f t} df.$$

\$\$

For Fourier Series:

\$\$

$$x(t) = \sum_{k=-\infty}^{\infty} c_k e^{jk\omega_0 t},$$

$\Leftrightarrow$

$$c_k = \frac{1}{T} \int_{t_0}^{t_0+T} x(t) e^{-jk\omega_0 t} dt,$$

$\Leftrightarrow$

$$\omega_0 = \frac{2\pi}{T}.$$

\$\$

Exam warning: Many formulas differ by factors of 2π if a book uses angular frequency ω instead of ordinary frequency f . Your supplied CFT questions use f **and** $e^{-j2\pi f t}$, so use the formulas in this document.

0. What the supplied previous questions tell us about the lab-test style

The supplied historical questions cover these patterns:

1. **Parseval verification** for a piecewise signal using numerical CFT and `trapz/trapezoid`.
2. **Recover component frequencies** of a complicated trigonometric signal from its CFT, then reconstruct the signal.
3. **Time shift** of a Gaussian and numerical verification of magnitude/phase properties using MSE.
4. **Modulation/frequency shift + time compression** and verification of the combined CFT property.
5. **Differentiation property**, including first, second, and third derivatives, magnitude/phase comparison, and MSE.
6. **Image denoising using Fourier Transform row by row**, identifying strong noise frequencies and suppressing them.
7. Your current offline work additionally gives:
 - complex Fourier-Series coefficients and reconstruction,
 - magnitude/phase of FS coefficients,
 - 2D CFT,
 - separable numerical integration,
 - frequency-domain high-pass filtering,
 - inverse 2D CFT.

So the safest preparation is **not** memorizing one assignment. You should be able to:

- generate/modify signals,
- numerically calculate FS/CFT,
- predict a transformed spectrum from a property,
- compare measured vs theoretical results,
- plot magnitude/phase,
- calculate MSE/energy,
- manipulate spectra,
- reconstruct signals/images,
- debug array shapes quickly.

1. The 30-minute lab-test workflow

When you receive a problem, do this in order.

Step 1 - Identify the domain

Ask:

- Periodic signal? -> **Fourier Series**.
- Nonperiodic/finite-window continuous-time signal? -> **1D CFT**.
- Image? -> **2D CFT**.

Step 2 - Write the mathematical operation before coding

Examples:

- Shift right by t_0 : $y(t) = x(t - t_0)$.
- Reverse: $y(t) = x(-t)$.
- Compress by $a > 1$: $y(t) = x(at)$.
- Frequency shift by f_0 : $y(t) = x(t)e^{j2\pi f_0 t}$.
- Differentiate: $y(t) = dx/dt$.
- Filter: $y = x * h$.

Step 3 - Write the theoretical property

Example:

$$y(t) = x(t - t_0) \implies Y(f) = X(f)e^{-j2\pi f t_0}.$$

Step 4 - Generate both signals numerically

Do **not** fake a continuous shift by moving array indices unless the question explicitly asks for a discrete circular shift. Evaluate the function at the modified argument:

```
x = signal(t)

y = signal(t - t0)
```

Step 5 - Calculate the transforms

Use the numerical CFT/FS framework. If FFT is forbidden, **do not use** `np.fft`.

Step 6 - Calculate the theoretical/predicted transform

Example:

```
Y_pred = X * np.exp(-1j * 2*np.pi*f*t0)
```

Step 7 - Plot what the question asks

Usually:

- original signals,
- magnitude spectra,
- phase spectra,
- reconstruction.

Step 8 - Quantify verification

```
mse = np.mean(np.abs(measured - predicted)**2)
```

For phase, use the special phase notes later in this document.

2. Master Python/NumPy toolbox

2.1 Imports

```
import numpy as np  
  
import matplotlib.pyplot as plt
```

2.2 Create a sampled axis

```
t = np.linspace(-5, 5, 2000)
```

```
f = np.linspace(-10, 10, 1000)
```

General form:

```
np.linspace(start, stop, number_of_samples)
```

Spacing:

```
dt = t[1] - t[0]
```

```
df = f[1] - f[0]
```

2.3 Element-wise signal operations

```
x = np.sin(2*np.pi*3*t)
```

```
y = np.exp(-t**2)
```

```
z = x + y
```

All operations are element-wise.

2.4 Complex numbers

```
z = 3 + 4j
```

```
np.abs(z)      # magnitude = 5
```

```
np.angle(z)    # phase in radians

z.real

z.imag

np.conj(z)
```

For arrays, the same functions operate element-wise.

2.5 Complex exponential

```
wave = np.exp(1j * 2*np.pi*f0*t)
```

represents

$$e^{j2\pi f_0 t}.$$

2.6 Numerical integration

1D

```
integral = np.trapezoid(values, t)
```

means

$$\int values(t) dt.$$

Along an axis

If `A.shape == (Y, X)`:

```
np.trapezoid(A, x, axis=1)
```

integrates along x , removing the x axis:

$$(Y, X) \rightarrow (Y,).$$

If `A.shape == (V, Y, U)`:

```
np.trapezoid(A, y, axis=1)
```

produces

$$(V, U).$$

Rule: the axis you integrate over disappears.

2.7 `np.outer`

```
phase = np.outer(f, t)
```

If:

```
f.shape = (F,)
```

```
t.shape = (T,)
```

then:

```
phase.shape = (F, T)
```

and

```
kernel = np.exp(-1j * 2*np.pi * np.outer(f, t))
```

contains $e^{-j2\pi f_i t_j}$ for every frequency/time pair.

2.8 Broadcasting with `None`

If:

```
I.shape      = (Y,X)
cosx.shape = (U,X)
```

then:

```
I[:, None, :].shape
```

is

```
(Y,1,X)
```

and:

```
cosx[None, :, :].shape
```

is

```
(1,U,X)
```

Multiplication broadcasts:

$$(Y, 1, X)(1, U, X) \rightarrow (Y, U, X).$$

This is the core shape trick in your 2D CFT.

2.9 Piecewise signals

Boolean-mask method - safest in lab

```
def signal(t):  
  
    x = np.zeros_like(t, dtype=float)  
  
    m1 = (t >= -2) & (t < 0)  
  
    x[m1] = t[m1] + 2  
  
    m2 = (t >= 0) & (t <= 2)  
  
    x[m2] = 2 - t[m2]  
  
    return x
```

np.where

```
x = np.where(np.abs(t) <= 1, 1.0, 0.0)
```

Do not write Python `and / or` with NumPy masks. Use:

```
& # and
```

```
| # or  
~ # not
```

and put every comparison in parentheses.

2.10 Numerical derivative

If the problem gives only sampled data:

```
dx = np.gradient(x, t)
```

But if the question gives an analytical function and asks you to **derive** the derivative, derive it mathematically and generate that expression. This is usually more accurate for property verification.

2.11 MSE

For real arrays:

```
mse = np.mean((a - b)**2)
```

For complex arrays:

```
mse = np.mean(np.abs(a - b)**2)
```

Magnitude MSE:

```
mse_mag = np.mean((np.abs(A) - np.abs(B))**2)
```

2.12 Useful comparisons

```
np.isclose(a, b)

np.allclose(A, B, rtol=1e-3, atol=1e-6)

np.all(np.isfinite(A))
```

2.13 Find a maximum / spectral peak

```
idx = np.argmax(magnitude)

peak_frequency = f[idx]
```

Top several values:

```
idx = np.argsort(magnitude)[-5:]
```

Better: only local peaks

```
peaks = np.where(

    (magnitude[1:-1] > magnitude[:-2]) &

    (magnitude[1:-1] > magnitude[2:]))

)[0] + 1
```

Then restrict to positive frequencies:

```
peaks = peaks[f[peaks] > 0]
```

2.14 Shift an array vs shift a continuous signal

`np.roll` shifts **samples** and wraps around:

```
shifted_samples = np.roll(x, 10)
```

This is **not** normally how you should implement

$$y(t) = x(t - t_0).$$

For a function generator, use:

```
y = signal(t - t0)
```

2.15 Interpolate a complex spectrum

Properties such as

$$X\left(\frac{f - f_0}{a}\right)$$

may ask for spectrum values between your sampled frequency bins.

```
def interp_complex(f_new, f, X):  
  
    real = np.interp(f_new, f, X.real, left=0.0, right=0.0)  
  
    imag = np.interp(f_new, f, X.imag, left=0.0, right=0.0)
```

```
return real + 1j*imag
```

Use this rather than interpolating phase directly.

3. Plotting cheat sheet

3.1 Plot a signal

```
plt.plot(t, x)

plt.xlabel("t")

plt.ylabel("x(t)")

plt.title("Signal")

plt.grid(True)

plt.show()
```

3.2 Magnitude spectrum

```
plt.plot(f, np.abs(X))

plt.xlabel("Frequency f")

plt.ylabel("|X(f)|")

plt.grid(True)

plt.show()
```

3.3 Phase spectrum

```
phase = np.angle(X)

plt.plot(f, phase)

plt.xlabel("Frequency f")

plt.ylabel("Phase (rad)")

plt.grid(True)

plt.show()
```

Unwrapped phase:

```
phase = np.unwrap(np.angle(X))
```

Use unwrapping for visualization of a linear phase trend, but understand the warning in the phase-error section.

3.4 FS coefficient spectrum

```
ns = np.array(sorted(fs.coeffs.keys()))

cs = np.array([fs.coeffs[n] for n in ns])

plt.stem(ns, np.abs(cs))

plt.xlabel("Harmonic k")

plt.ylabel("|c_k|")
```

```
plt.show()
```

Phase:

```
plt.stem(ns, np.angle(cs))  
  
plt.xlabel("Harmonic k")  
  
plt.ylabel("arg(c_k)")  
  
plt.show()
```

3.5 Image

```
plt.imshow(image, cmap='gray')  
  
plt.axis('off')  
  
plt.show()
```

3.6 2D magnitude spectrum

```
magnitude = np.sqrt(real**2 + imag**2)  
  
plt.imshow(np.log1p(magnitude), cmap='gray')  
  
plt.colorbar()  
  
plt.show()
```

`np.log1p(magnitude)` is numerically equivalent to `np.log(1 + magnitude)` .

3.7 2D phase spectrum

Your current assignment stores real and imaginary parts separately, so use:

```
phase = np.arctan2(imag, real)

plt.imshow(phase, cmap='twilight')

plt.colorbar()

plt.show()
```

`np.arctan2(imag, real)` is the phase of `real + j*imag` without needing complex arithmetic.

4. Your Task 1 FourierEpicycles class - what each function is for

Your current class conceptually contains:

```
fs = FourierEpicycles(t, signal, n_harmonics=N)
```

4.1 Constructor

Use when you have **one complete period** sampled on `t`.

Stores:

- `fs.t`
- `fs.signal`
- `fs.N`
- `fs.T`
- `fs.omega`
- `fs.coeffs`

Fundamental frequency:

$$\omega_0 = \frac{2\pi}{T}.$$

4.2 calculate_cn(n)

Use when the question asks:

- one Fourier coefficient,
- harmonic magnitude,
- harmonic phase,
- contribution of a specific harmonic.

```
c3 = fs.calculate_cn(3)

print(np.abs(c3))

print(np.angle(c3))
```

It calculates:

$$c_n = \frac{1}{T} \int_T x(t) e^{-jn\omega_0 t} dt.$$

4.3 calculate_all_coefficients()

Must be called before reconstruction if `self.coeffs` is empty.

```
fs.calculate_all_coefficients()
```

Stores all:

$$n = -N, \dots, 0, \dots, N.$$

Total coefficients:

$$2N + 1.$$

4.4 approximate(t)

Use to reconstruct the signal:

```
one_point = fs.approximate(0.5)

whole_curve = fs.approximate(t_dense)
```

Calculates:

$$\hat{x}_N(t) = \sum_{n=-N}^N c_n e^{jn\omega_0 t}.$$

Shape rule:

```
scalar t -> scalar/0-D result

(M,) t   -> (M,) result
```

4.5 Fast FS exam pattern

```
t = np.linspace(0, T, 2000)

x = ... # one full period

fs = FourierEpicyles(t, x, n_harmonics=20)

fs.calculate_all_coefficients()

ns = np.array(sorted(fs.coeffs.keys()))

cs = np.array([fs.coeffs[k] for k in ns])
```

```
mag = np.abs(cs)

phase = np.angle(cs)


x_hat = fs.approximate(t)
```

5. Fourier Series properties - master table

Assume

$$x(t) \leftrightarrow c_k, \quad \omega_0 = \frac{2\pi}{T}.$$

5.1 Linearity

$$ax(t) + by(t) \leftrightarrow ac_k + bd_k.$$

Python check:

```
pred = a*C + b*D
```

5.2 Time shift

$$y(t) = x(t - t_0)$$

then

$$\boxed{d_k = c_k e^{-jk\omega_0 t_0}}.$$

Consequences:

$$\begin{aligned} |d_k| &= |c_k|, \\ \angle d_k &= \angle c_k - k\omega_0 t_0. \end{aligned}$$

Numerically:

```
pred = C * np.exp(-1j * k * omega0 * t0)
```

5.3 Time reversal

$$y(t) = x(-t)$$

then

$$d_k = c_{-k}.$$

So reversing time reverses the coefficient sequence.

5.4 Conjugation

$$y(t) = x^*(t)$$

then

$$d_k = c_{-k}^*.$$

For a **real** signal $x(t) = x^*(t)$:

$$c_{-k} = c_k^*.$$

Therefore for real signals:

- magnitude spectrum is even,
 - coefficient phase is odd (where magnitude is nonzero).
-

5.5 Frequency/harmonic shift by complex modulation

$$y(t) = x(t)e^{jm\omega_0 t}$$

then

$$d_k = c_{k-m}.$$

Interpretation: all FS coefficients shift by m harmonic positions.

5.6 Multiply by cosine

Since

$$\cos(m\omega_0 t) = \frac{1}{2}e^{jm\omega_0 t} + \frac{1}{2}e^{-jm\omega_0 t},$$

$$y(t) = x(t) \cos(m\omega_0 t)$$

gives

$$d_k = \frac{1}{2}c_{k-m} + \frac{1}{2}c_{k+m}.$$

This creates **two shifted copies** of the spectrum.

5.7 Time scaling

For $a > 0$:

$$y(t) = x(at).$$

New period:

$$T_y = \frac{T}{a},$$

new fundamental angular frequency:

$$\omega_{0y} = a\omega_0.$$

When expressed using this new fundamental frequency, the coefficient values remain:

$$d_k = c_k.$$

For $a < 0$, scaling includes reversal, so with $\omega_{0y} = |a|\omega_0$:

$$d_k = c_{-k}.$$

Important intuition: compression in time increases the fundamental frequency. The same harmonic weights are now attached to faster harmonics.

5.8 Differentiation

$$y(t) = \frac{dx(t)}{dt}$$

then

$$d_k = jk\omega_0 c_k.$$

For the r -th derivative:

$$d_k = (jk\omega_0)^r c_k.$$

Consequences:

- DC coefficient becomes zero because $k = 0$.
- high harmonics are amplified more because of $|k|$.

5.9 Integration / periodic antiderivative

If

$$y'(t) = x(t),$$

then for $k \neq 0$:

$$d_k = \frac{c_k}{jk\omega_0}.$$

But there are two important issues:

1. d_0 is an arbitrary integration constant.
2. A periodic antiderivative exists only if the input has zero average:

$$c_0 = 0.$$

5.10 Multiplication in time

If

$$z(t) = x(t)y(t),$$

then the FS coefficient sequence is discrete convolution:

$$z_k = \sum_{m=-\infty}^{\infty} c_m d_{k-m}.$$

5.11 Periodic convolution

If periodic convolution is defined as

$$z(t) = \int_T x(\tau)y(t - \tau) d\tau,$$

then

$$z_k = Tc_kd_k.$$

If your instructor defines convolution with an extra $1/T$ normalization, then the T factor disappears. Always check the convention in the question.

5.12 Parseval / average power

$$\frac{1}{T} \int_T |x(t)|^2 dt = \sum_{k=-\infty}^{\infty} |c_k|^2$$

The left side is average power over one period.

Numerically with a truncated FS:

```
power_time = (1/fs.T) * np.trapezoid(np.abs(x)**2, t)

power_fs = sum(np.abs(c)**2 for c in fs.coeffs.values())
```

With finite N , they are only approximately equal unless the signal is completely represented by the retained harmonics.

5.13 DC / average value

$$c_0 = \frac{1}{T} \int_T x(t) dt.$$

So `calculate_cn(0)` gives the average value of a scalar real periodic signal.

5.14 Symmetry shortcuts for real signals

Real and even $x(t)$

- c_k is real,
- $c_{-k} = c_k$.

Real and odd $x(t)$

- c_k is purely imaginary,
- $c_{-k} = -c_k$.

General real $x(t)$

$$c_{-k} = c_k^*.$$

6. A reusable 1D numerical CFT template

This is the most important template for a surprise 1D CFT question.

```
class CFT1D:

    def __init__(self, t, signal, f):

        self.t = np.asarray(t)

        self.signal = np.asarray(signal)

        self.f = np.asarray(f)

    def transform(self):

        # kernel shape: (F,T)

        kernel = np.exp(-1j * 2*np.pi * np.outer(self.f, self.t))

        # signal[None,:] shape: (1,T)
```

```

        # broadcast -> (F,T), integrate over time axis

        X = np.trapezoid(

            self.signal[None, :] * kernel,

            self.t,

            axis=1

        )

        return X

```

You can also write:

```

X = np.trapezoid(self.signal * kernel, self.t, axis=1)

```

because `(T,)` broadcasts across `(F,T)`.

7. Reusable 1D inverse CFT template

```

def inverse_cft(f, X, t_recon):

    # kernel shape: (T,F)

    kernel = np.exp(1j * 2*np.pi * np.outer(t_recon, f))

    x_hat = np.trapezoid(

        X[None, :] * kernel,

        f,

        axis=1

```

```
)  
  
return x_hat
```

For a theoretically real signal:

```
x_hat_real = x_hat.real
```

Small numerical imaginary residuals can occur due to finite integration and floating-point error.

8. Continuous Fourier Transform properties - master table

Assume

$$x(t) \leftrightarrow X(f)$$

with

$$X(f) = \int x(t) e^{-j2\pi ft} dt.$$

8.1 Linearity

$$ax(t) + by(t) \leftrightarrow aX(f) + bY(f)$$

8.2 Time shift

$$y(t) = x(t - t_0)$$

$$Y(f) = X(f) e^{-j2\pi ft_0}$$

Consequences:

$$|Y(f)| = |X(f)|,$$

$$\angle Y(f) = \angle X(f) - 2\pi ft_0 \pmod{2\pi}.$$

This exact property appeared in a supplied previous question.

8.3 Time reversal

$$y(t) = x(-t)$$

$$\boxed{Y(f) = X(-f)}.$$

8.4 Time scaling

$$y(t) = x(at)$$

$$\boxed{Y(f) = \frac{1}{|a|} X\left(\frac{f}{a}\right)}$$

If $|a| > 1$:

- time signal is compressed,
- spectrum becomes wider,
- magnitude gets factor $1/|a|$.

If $0 < |a| < 1$:

- time signal expands,
- spectrum becomes narrower.

If $a < 0$, the formula automatically includes spectral reversal through f/a .

8.5 Frequency shift / modulation

$$y(t) = x(t)e^{j2\pi f_0 t}$$

$$\boxed{Y(f) = X(f - f_0)}.$$

The entire spectrum shifts right by f_0 .

For negative exponential:

$$x(t)e^{-j2\pi f_0 t} \leftrightarrow X(f + f_0).$$

8.6 Multiply by cosine

$$y(t) = x(t) \cos(2\pi f_0 t)$$

Since

$$\cos(2\pi f_0 t) = \frac{1}{2} e^{j2\pi f_0 t} + \frac{1}{2} e^{-j2\pi f_0 t},$$

$$Y(f) = \frac{1}{2} X(f - f_0) + \frac{1}{2} X(f + f_0)$$

This creates **two shifted copies**.

8.7 Combined time compression + frequency shift

If

$$y(t) = x(at) e^{j2\pi f_0 t},$$

then

$$Y(f) = \frac{1}{|a|} X\left(\frac{f - f_0}{a}\right)$$

This exact form appeared in the supplied July 2025 question.

8.8 Combined scaling + time shift

If

$$y(t) = x(a(t - t_0)),$$

first define $g(t) = x(at)$, then shift g by t_0 :

$$Y(f) = \frac{1}{|a|} X\left(\frac{f}{a}\right) e^{-j2\pi f t_0}$$

If modulation is also added:

$$y(t) = x(a(t - t_0)) e^{j2\pi f_0 t},$$

then

$$Y(f) = \frac{1}{|a|} X\left(\frac{f-f_0}{a}\right) e^{-j2\pi(f-f_0)t_0}$$

8.9 Conjugation

$$y(t) = x^*(t)$$

$$Y(f) = X^*(-f).$$

For real $x(t)$:

$$X(-f) = X^*(f).$$

Therefore:

- $|X(f)|$ is even,
- phase is odd where the magnitude is nonzero.

8.10 Differentiation in time

$$y(t) = \frac{dx}{dt}$$

$$Y(f) = j2\pi f X(f)$$

For the n -th derivative:

$$\mathcal{F}\left\{\frac{d^n x}{dt^n}\right\} = (j2\pi f)^n X(f)$$

This property appeared directly in a supplied previous question.

8.11 Multiply by time

Differentiate $X(f)$:

$$\frac{dX}{df} = -j2\pi \mathcal{F}\{tx(t)\}.$$

Therefore:

$$\mathcal{F}\{tx(t)\} = \frac{j}{2\pi} \frac{dX(f)}{df}$$

And more generally:

$$\mathcal{F}\{t^n x(t)\} = \left(\frac{j}{2\pi}\right)^n \frac{d^n X}{df^n}$$

8.12 Convolution theorem

If

$$y(t) = x(t) * h(t),$$

then

$$Y(f) = X(f)H(f).$$

This is the central LTI/filtering property.

Numerical time convolution if allowed:

```
y = np.convolve(x, h, mode='same') * dt
```

The `* dt` approximates the continuous convolution integral.

Then compare:

```
Y_pred = X * H
```

8.13 Multiplication theorem

If

$$y(t) = x(t)h(t),$$

then

$$Y(f) = X(f) * H(f)$$

where the convolution is now over frequency.

8.14 Parseval / energy theorem

$$\int_{-\infty}^{\infty} |x(t)|^2 dt = \int_{-\infty}^{\infty} |X(f)|^2 df$$

Numerically:

```
E_time = np.trapezoid(np.abs(x)**2, t)

E_freq = np.trapezoid(np.abs(X)**2, f)

print(E_time, E_freq, abs(E_time - E_freq))
```

Because your numerical windows are finite, exact equality is not expected. Use a sufficiently wide/dense time and frequency range.

8.15 Cross-Parseval

$$\int x(t)y^*(t)dt = \int X(f)Y^*(f)df$$

8.16 Duality

With this f convention:

$$x(t) \leftrightarrow X(f)$$

implies

$$X(t) \leftrightarrow x(-f).$$

8.17 Area / DC relationships

At $f = 0$:

$$X(0) = \int x(t) dt.$$

At $t = 0$ in the inverse:

$$x(0) = \int X(f) df.$$

These are useful sanity checks.

8.18 Integration in time - caution

If $y'(t) = x(t)$, then away from DC:

$$Y(f) = \frac{X(f)}{j2\pi f}.$$

At $f = 0$, integration has a DC/distribution issue. In lab coding, if the signal has zero DC and the question ignores the $f = 0$ distribution term, treat:

$$Y(f) \approx X(f)/(j2\pi f), \quad f \neq 0$$

and handle $f == 0$ separately.

9. Real/even/odd CFT symmetry table

Real $x(t)$

$$X(-f) = X^*(f).$$

Therefore:

- real part of X is even,
- imaginary part is odd,
- magnitude is even,
- phase is odd modulo wrapping.

Real and even $x(t)$

$X(f)$ is real and even.

Real and odd $x(t)$

$X(f)$ is imaginary and odd.

Purely imaginary and even $x(t)$

$X(f)$ is imaginary and even.

Purely imaginary and odd $x(t)$

$X(f)$ is real and odd.

10. Common transform pairs useful for sanity checks

These are analytical references; your numerical CFT will only approximate them because of finite windows.

10.1 Gaussian

$$x(t) = e^{-at^2}, \quad a > 0$$

$$X(f) = \sqrt{\frac{\pi}{a}} e^{-\pi^2 f^2 / a}$$

Special normalized pair:

$$e^{-\pi t^2} \leftrightarrow e^{-\pi f^2}.$$

10.2 Rectangle

If

$$\text{rect}(t/T) = 1 \quad \text{for } |t| \leq T/2,$$

then

$$\text{rect}(t/T) \leftrightarrow T \text{ sinc}(Tf)$$

where NumPy uses

$$\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}.$$

So:

```
X_theory = T * np.sinc(T*f)
```

10.3 Triangle

A triangular pulse transforms to a sinc-squared form:

$$\text{tri}(t/T) \leftrightarrow T \text{sinc}^2(Tf)$$

with exact scaling depending on your triangle definition.

10.4 Cosine and sine

The infinite-duration cosine has impulses:

$$\cos(2\pi f_0 t) \leftrightarrow \frac{1}{2} \delta(f - f_0) + \frac{1}{2} \delta(f + f_0).$$

$$\sin(2\pi f_0 t) \leftrightarrow \frac{1}{2j} [\delta(f - f_0) - \delta(f + f_0)].$$

In numerical finite-window CFT, you do **not** see ideal delta functions. You see narrow peaks around $\pm f_0$.

11. Property-verification code recipes

11.1 Time shift

Suppose:

```
def g(t):  
  
    return np.exp(-t**2)
```

```
x = g(t)

y = g(t - t0)
```

Calculate:

```
X = CFT1D(t, x, f).transform()

Y = CFT1D(t, y, f).transform()

Y_pred = X * np.exp(-1j * 2*np.pi*f*t0)
```

Magnitude verification:

```
mse_mag = np.mean((np.abs(Y) - np.abs(X))**2)
```

Complex-spectrum verification - often better:

```
mse_complex = np.mean(np.abs(Y - Y_pred)**2)
```

11.2 Time reversal

```
x = g(t)

y = g(-t)

X = CFT1D(t, x, f).transform()
```

```
Y = CFT1D(t, y, f).transform()

X_minus_f = interp_complex(-f, f, X)

Y_pred = X_minus_f
```

11.3 Time scaling

```
a = 3

x = g(t)

y = g(a*t)

X = CFT1D(t, x, f).transform()

Y = CFT1D(t, y, f).transform()

X_scaled = interp_complex(f/a, f, X)

Y_pred = (1/abs(a)) * X_scaled
```

Important: your time interval must still capture the important support of both signals.

11.4 Modulation / frequency shift

```
f0 = 4

x = g(t)
```

```

y = x * np.exp(1j * 2*np.pi*f0*t)

X = CFT1D(t, x, f).transform()

Y = CFT1D(t, y, f).transform()

Y_pred = interp_complex(f - f0, f, X)

```

11.5 Combined modulation + compression

```

a = 10

f0 = 10

y = g(a*t) * np.exp(1j * 2*np.pi*f0*t)

X = CFT1D(t, g(t), f).transform()

Y = CFT1D(t, y, f).transform()

arg = (f - f0) / a

Y_pred = (1/abs(a)) * interp_complex(arg, f, X)

```

The property is:

$$Y(f) = \frac{1}{|a|} X\left(\frac{f - f_0}{a}\right).$$

11.6 Differentiation

First derivative:

```
X = CFT1D(t, x, f).transform()

Y1 = CFT1D(t, dx, f).transform()

Y1_pred = (1j * 2*np.pi*f) * X
```

Second:

```
Y2_pred = (1j * 2*np.pi*f)**2 * X
```

Third:

```
Y3_pred = (1j * 2*np.pi*f)**3 * X
```

11.7 Parseval

```
X = CFT1D(t, x, f).transform()

E_time = np.trapezoid(np.abs(x)**2, t)

E_freq = np.trapezoid(np.abs(X)**2, f)

print("Time energy:", E_time)
```

```
print("Freq energy:", E_freq)

print("Absolute difference:", abs(E_time - E_freq))

print("Relative error:", abs(E_time-E_freq) / max(E_time, 1e-12))
```

11.8 Convolution theorem

```
dt = t[1] - t[0]

y = np.convolve(x, h, mode='same') * dt

X = CFT1D(t, x, f).transform()
H = CFT1D(t, h, f).transform()
Y = CFT1D(t, y, f).transform()

Y_pred = X * H
```

There may be boundary/truncation error because `mode='same'` and a finite time window are approximations.

12. Phase: the part most likely to create false errors

12.1 Why direct phase subtraction can fail

`np.angle()` returns phase in approximately:

$$[-\pi, \pi].$$

But π and $-\pi$ represent the same direction.

Example:

```
measured  = +3.13 rad  
predicted = -3.15 rad
```

Direct difference is about 6.28, even though the true angular difference is almost zero.

12.2 Wrapped phase difference - best numerical check

```
phase_meas = np.angle(Y)  
phase_pred = np.angle(Y_pred)  
  
phase_diff = np.angle(  
    np.exp(1j * (phase_meas - phase_pred))  
)  
  
mse_phase = np.mean(phase_diff**2)
```

This forces the phase error back to $[-\pi, \pi]$.

12.3 Ignore phase where magnitude is almost zero

Phase is physically meaningless when magnitude is approximately zero.

```
threshold = 1e-3 * np.max(np.abs(Y))  
mask = (np.abs(Y) > threshold) & (np.abs(Y_pred) > threshold)
```

```
mse_phase = np.mean(phase_diff[mask]**2)
```

Exam-simple version

If the instructor explicitly gives a formula such as

```
np.mean((phase_Y - predicted_phase)**2)
```

follow the requested formula. But if the result looks strangely large, phase wrapping is the first thing to suspect.

13. Frequency-component detection

Suppose a signal is a sum of sinusoids. Its finite-window numerical CFT will show peaks near their frequencies.

```
X = CFT1D(t, x, f).transform()  
  
mag = np.abs(X)
```

Use positive frequency side:

```
mask = f > 0  
  
fp = f[mask]  
  
mp = mag[mask]
```

Find local maxima:

```

peak_idx = np.where(
    (mp[1:-1] > mp[:-2]) &
    (mp[1:-1] > mp[2:]))
)[0] + 1

```

Top three peaks:

```

top = peak_idx[np.argsort(mp[peak_idx])[-3:]]

found_frequencies = np.sort(fp[top])

print(found_frequencies)

```

Why local peaks? `np.argsort(magnitude)` alone can select three neighboring samples belonging to the same broad peak.

14. Current Task 2 2D CFT - function-use chart

ContinuousImage(path)

Use to:

- load grayscale image,
- normalize image,
- construct spatial axes `x`, `y`.

```

img = ContinuousImage("pikachu.png")

```

Important shapes:

```
img.image.shape = (Y,X)
```

```
len(img.y) = Y
```

```
len(img.x) = X
```

CFT2D(img)

```
cft = CFT2D(img)
```

Constructs:

- `cft.u` : horizontal spatial frequencies,
- `cft.v` : vertical spatial frequencies.

compute_cft()

```
real, imag = cft.compute_cft()
```

Both shapes:

```
(V,U) = image.shape = (Y,X)
```

Magnitude:

```
mag = np.sqrt(real**2 + imag**2)
```

Phase:

```
phase = np.arctan2(imag, real)
```

plot_magnitude()

```
cft.plot_magnitude()
```

Debug spectrum. Low-frequency energy should be concentrated near the center for a normal image.

FrequencyFilter.high_pass()

```
filt = FrequencyFilter()

real_hp, imag_hp = filt.high_pass(real, imag, cutoff=15)
```

Removes a central disk of low frequencies.

InverseCFT2D(...)

```
inv = InverseCFT2D(
    real_hp, imag_hp,
    cft.u, cft.v,
    img.x, img.y
)
```

```
recon = inv.reconstruct()
```

Returns a real (Y,X) array.

15. Shape map for the current 2D CFT

Forward transform:

$I(y,x)$	(Y,X)
$\cos(2\pi ux), \sin(2\pi ux)$	(U,X)
broadcast	(Y,U,X)
integrate x	(Y,U)
$\cos(2\pi vy), \sin(2\pi vy)$	(V,Y)
broadcast	(V,Y,U)
integrate y	(V,U)

Inverse transform:

$\text{real}(v,u), \text{imag}(v,u)$	(V,U)
$\cos(2\pi vy), \sin(2\pi vy)$	(V,Y)
broadcast	(V,U,Y)

integrate v	(U, Y)
cos(2πux), sin(2πux)	(U, X)
broadcast	(U, Y, X)
integrate u	(Y, X)

Golden rule: before integrating over a variable, that variable must still exist as an axis.

16. 2D CFT properties that can become lab-test modifications

Assume

$$I(x, y) \leftrightarrow F(u, v).$$

16.1 Spatial shift

$$J(x, y) = I(x - x_0, y - y_0)$$

$$G(u, v) = F(u, v)e^{-j2\pi(ux_0 + vy_0)}$$

Magnitude remains unchanged; phase changes linearly.

16.2 Spatial reversal

$$I(-x, -y) \leftrightarrow F(-u, -v).$$

Horizontal flip only:

$$I(-x, y) \leftrightarrow F(-u, v).$$

Vertical flip only:

$$I(x, -y) \leftrightarrow F(u, -v).$$

16.3 2D modulation / frequency shift

$$I(x, y)e^{j2\pi(u_0x+v_0y)}$$

$$F(u - u_0, v - v_0).$$

16.4 2D scaling

$$J(x, y) = I(ax, by)$$

$$G(u, v) = \frac{1}{|ab|} F\left(\frac{u}{a}, \frac{v}{b}\right)$$

16.5 2D convolution

$$I * h \leftrightarrow F(u, v)H(u, v).$$

This is image filtering.

16.6 Partial derivative / edges

$$\mathcal{F}\left\{\frac{\partial I}{\partial x}\right\} = j2\pi u F(u, v)$$

$$\mathcal{F}\left\{\frac{\partial I}{\partial y}\right\} = j2\pi v F(u, v)$$

Laplacian:

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

$$\mathcal{F}\{\nabla^2 I\} = -4\pi^2(u^2 + v^2)F(u, v)$$

This naturally emphasizes high spatial frequencies/edges.

16.7 Real-image conjugate symmetry

For real $I(x, y)$:

$$F(-u, -v) = F^*(u, v).$$

Numerical check before filtering:

```
real_err = np.max(np.abs(real - real[::-1, ::-1]))  
  
imag_err = np.max(np.abs(imag + imag[::-1, ::-1]))
```

These should be small for symmetric grids, up to numerical error.

17. Common 2D frequency filters you may be asked to modify

Your current `high_pass` is index-radius based. These generic masks use the same idea.

```
rows, cols = real.shape  
  
cy, cx = rows // 2, cols // 2  
  
Y, X = np.ogrid[:rows, :cols]  
  
r = np.sqrt((Y - cy)**2 + (X - cx)**2)
```

17.1 Low-pass

Keep center, remove high frequencies:

```
mask = r <= cutoff  
  
real_lp = real * mask  
  
imag_lp = imag * mask
```

Expected result after inverse: blurred/smoothed image.

17.2 High-pass

```
mask = r > cutoff  
  
real_hp = real * mask  
  
imag_hp = imag * mask
```

Expected: edges/fine details.

17.3 Band-pass

```
mask = (r >= low) & (r <= high)  
  
real_bp = real * mask  
  
imag_bp = imag * mask
```

Keeps only a ring of spatial frequencies.

17.4 Band-stop

```
mask = ~((r >= low) & (r <= high))
```

Removes one frequency ring.

17.5 Notch filtering

If periodic noise creates isolated bright spectral peaks, zero a small neighborhood around each noise peak and its symmetric counterpart.

Conceptual helper:

```
def zero_disk(real, imag, row0, col0, radius):  
  
    rr, cc = np.ogrid[:real.shape[0], :real.shape[1]]  
  
    mask = (rr-row0)**2 + (cc-col0)**2 <= radius**2  
  
    real[mask] = 0  
  
    imag[mask] = 0
```

For a real image, suppress symmetric peak pairs to preserve conjugate symmetry.

18. Filtering intuition you should be able to explain verbally

Low frequencies

- average brightness/DC,
- smooth areas,
- slow spatial changes.

High frequencies

- edges,
- fine texture,
- abrupt brightness changes,
- often high-frequency noise.

Periodic stripe noise

Periodic stripes correspond to specific spatial frequencies, producing isolated or concentrated spectral peaks. A notch filter can remove those frequencies without deleting all high-frequency content.

19. Why finite numerical CFT does not look exactly like textbook CFT

Your computer uses finite time/spatial windows and finite samples.

Therefore:

1. textbook impulses become finite peaks,
2. spectral leakage may spread energy into nearby frequencies,
3. insufficient time window truncates signals,
4. insufficient frequency range harms inverse reconstruction/Parseval,
5. insufficient sample density introduces numerical integration error.

So a small nonzero MSE does not automatically mean your property is wrong.

20. PREVIOUS-YEAR QUESTION 1 - Parseval for the piecewise signal

Source: supplied `Online_B1B2_CSE220.pdf`.

The graph shows a symmetric piecewise function over $[-3, 3]$ with parabolic outer parts and straight inner parts. A natural expression matching the graph is:

$$x(t) = \begin{cases} (t+3)^2, & -3 \leq t < -1 \\ t+5, & -1 \leq t < 0 \\ 5-t, & 0 \leq t \leq 1 \\ (t-3)^2, & 1 < t \leq 3 \\ 0, & \text{otherwise} \end{cases}$$

Implementation

```
def x_func(t):  
  
    x = np.zeros_like(t, dtype=float)  
  
    m = (t >= -3) & (t < -1)  
  
    x[m] = (t[m] + 3)**2
```

```
m = (t >= -1) & (t < 0)
```

```
x[m] = t[m] + 5
```

```
m = (t >= 0) & (t <= 1)
```

```
x[m] = 5 - t[m]
```

```
m = (t > 1) & (t <= 3)
```

```
x[m] = (t[m] - 3)**2
```

```
return x
```

Use a sufficiently dense time axis, then calculate numerical CFT and:

```
E_time = np.trapezoid(np.abs(x)**2, t)
```

```
E_freq = np.trapezoid(np.abs(X)**2, f)
```

Expected:

$$E_{time} \approx E_{freq}.$$

Analytical time-energy sanity check

The function is even, so:

$$E = 2 \left[\int_0^1 (5-t)^2 dt + \int_1^3 (t-3)^4 dt \right].$$

This gives:

$$E = \frac{802}{15} \approx 53.4667.$$

Your numerical time-domain integral should be close to this.

Main trap

Parseval integrates over all frequency. If you choose a narrow frequency range, E_{freq} will be too low even if your CFT code is correct.

21. PREVIOUS-YEAR QUESTION 2 - Find the hidden component frequencies

Source: supplied `Online_C1C2_CSE220_FT.pdf`.

Given:

$$x(t) = 2 \sin(14\pi t) - \sin(2\pi t) (4 \sin(2\pi t) \sin(14\pi t) - 1).$$

Simplify:

$$x(t) = 2 \sin(14\pi t) + \sin(2\pi t) - 4 \sin^2(2\pi t) \sin(14\pi t).$$

Use:

$$\sin^2 \theta = \frac{1 - \cos 2\theta}{2}.$$

Then:

$$-4 \sin^2(2\pi t) \sin(14\pi t) = -2 \sin(14\pi t) + 2 \cos(4\pi t) \sin(14\pi t).$$

The first term cancels the original $2 \sin(14\pi t)$.

Use:

$$2 \sin A \cos B = \sin(A + B) + \sin(A - B).$$

Therefore:

$$2 \sin(14\pi t) \cos(4\pi t) = \sin(18\pi t) + \sin(10\pi t).$$

So:

$$x(t) = \sin(2\pi t) + \sin(10\pi t) + \sin(18\pi t)$$

which means frequencies:

$$\boxed{1 \text{ Hz}, 5 \text{ Hz}, 9 \text{ Hz}}.$$

All amplitudes are 1 and phases are 0 in the sine representation.

Numerical verification

```
x2 = (
    np.sin(2*np.pi*1*t) +
    np.sin(2*np.pi*5*t) +
    np.sin(2*np.pi*9*t)
)

print(np.mean((x - x2)**2))
```

Should be near floating-point zero.

The CFT magnitude should have symmetric peaks near:

$$\pm 1, \pm 5, \pm 9 \text{ Hz}.$$

22. PREVIOUS-YEAR QUESTION 3 - Gaussian time shift

Source: supplied CSE220_OnlineCFT_B1_B2.pdf .

Original:

$$x(t) = e^{-t^2}.$$

Shift:

$$y(t) = x(t - 1) = e^{-(t-1)^2}.$$

Property:

$$\boxed{Y(f) = X(f)e^{-j2\pi f}}.$$

Therefore:

$$|Y(f)| = |X(f)|$$

and

$$\angle Y(f) = \angle X(f) - 2\pi f$$

modulo phase wrapping.

Coding pattern

```
def gaussian(t, a=1, shift=0):
    return np.exp(-a * (t-shift)**2)

x = gaussian(t, 1, 0)
y = gaussian(t, 1, 1)

X = CFT1D(t, x, f).transform()
Y = CFT1D(t, y, f).transform()

Y_pred = X * np.exp(-1j*2*np.pi*f*1)
```

Best verification:

```
mse_complex = np.mean(np.abs(Y - Y_pred)**2)
```

Magnitude:

```
mse_mag = np.mean((np.abs(Y) - np.abs(X))**2)
```

23. PREVIOUS-YEAR QUESTION 4 - Time compression + phase/frequency shift

Source: supplied July 2025 CSE 220 CFT Online.pdf .

The question combines:

1. multiplication by a complex phase factor $e^{j2\pi f_0 t}$,
2. time compression $x(at)$.

If:

$$y(t) = x(at)e^{j2\pi f_0 t},$$

then:

$$Y(f) = \frac{1}{|a|} X\left(\frac{f - f_0}{a}\right)$$

For the supplied values:

$$a = 10, \quad f_0 = 10.$$

Expected magnitude:

$$|Y(f)| = \frac{1}{10} \left| X\left(\frac{f - 10}{10}\right) \right|$$

Expected phase:

$$\angle Y(f) = \angle X\left(\frac{f - 10}{10}\right)$$

where magnitude is nonzero.

Coding pattern

```
y = x_generator(a*t) * np.exp(1j*2*np.pi*f0*t)
```

```
X = CFT1D(t, x_generator(t), f).transform()
```

```
Y = CFT1D(t, y, f).transform()
```

```
arg = (f - f0) / a

X_at_arg = interp_complex(arg, f, X)

Y_pred = (1/abs(a)) * X_at_arg
```

Then compare `Y` directly with `Y_pred` or compare magnitude and wrapped phase.

24. PREVIOUS-YEAR QUESTION 5 - First, second, third derivatives

Source: supplied 220_online.pdf .

Given:

$$x(t) = 0.5 \cos(4t) + 0.5 \sin(6t).$$

Remember these arguments are in radians, so the sinusoidal frequencies in Hz are:

$$f_1 = \frac{4}{2\pi}, \quad f_2 = \frac{6}{2\pi}.$$

First derivative

$$x'(t) = -2 \sin(4t) + 3 \cos(6t)$$

and:

$$Y_1(f) = j2\pi f X(f).$$

Second derivative

$$x''(t) = -8 \cos(4t) - 18 \sin(6t)$$

$$Y_2(f) = (j2\pi f)^2 X(f).$$

Third derivative

$$x'''(t) = 32 \sin(4t) - 108 \cos(6t)$$

$$Y_3(f) = (j2\pi f)^3 X(f).$$

Code structure

```
x = 0.5*np.cos(4*t) + 0.5*np.sin(6*t)

y1 = -2*np.sin(4*t) + 3*np.cos(6*t)

y2 = -8*np.cos(4*t) - 18*np.sin(6*t)

y3 = 32*np.sin(4*t) - 108*np.cos(6*t)


X = CFT1D(t, x, f).transform()

Y1 = CFT1D(t, y1, f).transform()

Y2 = CFT1D(t, y2, f).transform()

Y3 = CFT1D(t, y3, f).transform()


P1 = (1j*2*np.pi*f) * X

P2 = (1j*2*np.pi*f)**2 * X

P3 = (1j*2*np.pi*f)**3 * X
```

Then calculate magnitude/phase MSE for (Y_1, P_1) , (Y_2, P_2) , (Y_3, P_3) .

25. PREVIOUS-YEAR QUESTION 6 - Row-by-row Fourier image denoising

Source: supplied `Online_A1A2_FT.zip` historical bundle.

The problem supplies a noisy 64x64 grayscale image and tells you to:

- apply FT row by row,

- identify frequencies responsible for noise,
- suppress them,
- inverse-transform rows,
- identify the secret letter.

The image contains strong vertical stripe interference. Vertical stripes mean brightness changes mainly as you move in the horizontal x direction, so **row-wise 1D transforms are appropriate**.

General solution strategy

Let each row be a 1D signal:

$$r_y(x).$$

For every row:

1. calculate $R_y(f)$,
2. inspect magnitude,
3. suppress strong noise-frequency pairs $\pm f_n$,
4. inverse CFT to reconstruct the row.

A useful way to identify globally strong stripe-noise frequencies is to average row spectra:

```
all_mag = []

for row in image:

    R = CFT1D(x_axis, row, f).transform()

    all_mag.append(np.abs(R))

mean_mag = np.mean(all_mag, axis=0)
```

Periodic interference appears as strong spectral peaks shared by many rows.

After zeroing/suppressing the noise bands, reconstruct each row with the inverse CFT.

Historical hidden-letter answer

For the supplied image bundle, suppressing the dominant symmetric stripe-frequency components reveals a **C-shaped letter**.

The exact numerical notch locations depend on how you define the row coordinate and frequency axis, so the robust method is to inspect the mean spectrum rather than hard-code a number from someone else's coordinate convention.

26. Templates for common signal modifications

Assume:

```
def x_func(t):  
    return np.exp(-t**2)
```

Shift right

$$x(t - t_0)$$

```
y = x_func(t - t0)
```

Shift left

$$x(t + t_0)$$

```
y = x_func(t + t0)
```

Reverse

$$x(-t)$$

```
y = x_func(-t)
```

Compress

$$x(at), |a| > 1$$

```
y = x_func(a*t)
```

Expand

$$x(at), 0 < |a| < 1$$

```
y = x_func(a*t)
```

Scale amplitude

$$Ax(t)$$

```
y = A*x_func(t)
```

Frequency shift

$$x(t)e^{j2\pi f_0 t}$$

```
y = x_func(t) * np.exp(1j*2*np.pi*f0*t)
```

Cosine modulation

$$x(t) \cos(2\pi f_0 t)$$

```
y = x_func(t) * np.cos(2*np.pi*f0*t)
```

Differentiate

Analytical if possible; otherwise:

```
y = np.gradient(x, t)
```

Add signals

```
y = x1 + x2
```

Multiply signals

```
y = x1 * x2
```

Convolve

```
y = np.convolve(x, h, mode='same') * (t[1]-t[0])
```

27. A universal CFT property-check harness

```
def complex_mse(A, B):  
    return np.mean(np.abs(A-B)**2)  
  
def magnitude_mse(A, B):  
    return np.mean((np.abs(A)-np.abs(B))**2)
```

```
def phase_mse(A, B):

    # Compare phases robustly and ignore almost-zero magnitudes

    pa = np.angle(A)

    pb = np.angle(B)

    diff = np.angle(np.exp(1j*(pa-pb)))

    threshold = 1e-3 * max(np.max(np.abs(A)), np.max(np.abs(B)))

    mask = (np.abs(A) > threshold) & (np.abs(B) > threshold)

    if not np.any(mask):

        return np.nan

    return np.mean(diff[mask]**2)
```

Then:

```
print("Complex MSE:", complex_mse(Y, Y_pred))

print("Magnitude MSE:", magnitude_mse(Y, Y_pred))

print("Phase MSE:", phase_mse(Y, Y_pred))
```

28. FS property-check harness using your class

Example: verify time shift.

```

T = 2*np.pi

t = np.linspace(0, T, 2000)

t0 = 0.7

# Example periodic signal

x = np.cos(2*t) + 0.5*np.sin(3*t)

y = np.cos(2*(t-t0)) + 0.5*np.sin(3*(t-t0))


fs_x = FourierEpicycles(t, x, 10)

fs_y = FourierEpicycles(t, y, 10)


fs_x.calculate_all_coefficients()

fs_y.calculate_all_coefficients()


for k in range(-10, 11):

    measured = fs_y.coeffs[k]

    predicted = fs_x.coeffs[k] * np.exp(-1j*k*fs_x.omega*t0)


    print(k, abs(measured-predicted))

```

For an exam, calculate an MSE over coefficient arrays instead of printing all errors.

29. PRACTICE SET - with answers

Try each before reading its answer.

Practice 1 - CFT time reversal

Given:

$$x(t) = e^{-(t-1)^2}.$$

Define:

$$y(t) = x(-t).$$

Questions

1. Write $y(t)$ explicitly.
2. Predict $Y(f)$ from $X(f)$.
3. How would you verify it numerically?

Answer

$$y(t) = e^{-(-t-1)^2} = e^{-(t+1)^2}.$$

$$\boxed{Y(f) = X(-f)}.$$

Numerically:

```
X_minus = interp_complex(-f, f, X)

Y_pred = X_minus

mse = np.mean(np.abs(Y-Y_pred)**2)
```

Practice 2 - Shift + differentiation

$$y(t) = \frac{d}{dt}x(t-2).$$

Find $Y(f)$ in terms of $X(f)$.

Answer

Shift first:

$$x(t - 2) \leftrightarrow X(f)e^{-j4\pi f}.$$

Differentiate:

$$Y(f) = j2\pi f X(f)e^{-j4\pi f}.$$

Order gives the same result if handled correctly.

Practice 3 - Modulation by cosine

$$y(t) = x(t) \cos(2\pi 5t).$$

Answer

$$Y(f) = \frac{1}{2} X(f - 5) + \frac{1}{2} X(f + 5).$$

Two half-amplitude shifted copies.

Practice 4 - Time compression

$$y(t) = x(4t).$$

If $X(f)$ has a strong peak at 2 Hz, where does the corresponding peak occur in $Y(f)$?

Answer

$$Y(f) = \frac{1}{4} X(f/4).$$

For the argument to equal 2:

$$f/4 = 2 \Rightarrow f = 8 \text{ Hz}.$$

Peak moves from 2 Hz to 8 Hz and its magnitude scales by 1/4.

Practice 5 - Negative time scaling

$$y(t) = x(-2t).$$

Answer

$$Y(f) = \frac{1}{2}X(-f/2).$$

Contains both reversal and compression.

Practice 6 - Parseval

You calculate:

```
E_time = 3.201
```

```
E_freq = 2.74
```

Your code seems correct. What should you check first?

Answer

Check:

1. frequency range too narrow,
2. too few frequency samples,
3. time range truncating the signal,
4. too few time samples.

Parseval is over infinite time/frequency domains; numerical integration only covers your finite windows.

Practice 7 - CFT derivative phase

For $f > 0$, multiplying $X(f)$ by $j2\pi f$ adds what phase?

Answer

For $f > 0$, $j2\pi f$ has phase:

$$+\frac{\pi}{2}.$$

For $f < 0$, $j2\pi f$ is negative imaginary and has phase approximately:

$$-\frac{\pi}{2}.$$

Magnitude is multiplied by $2\pi|f|$.

Practice 8 - FS derivative

If:

$$x(t) \leftrightarrow c_k,$$

what is the FS coefficient of $x'''(t)$?

Answer

$$d_k = (jk\omega_0)^3 c_k.$$

DC $k = 0$ becomes zero.

Practice 9 - FS shift

A signal has coefficient:

$$c_3 = 2e^{j\pi/4}.$$

It is shifted right by t_0 . What happens to $|c_3|$?

Answer

Nothing:

$$|d_3| = |c_3| = 2.$$

Only phase changes:

$$\angle d_3 = \frac{\pi}{4} - 3\omega_0 t_0.$$

Practice 10 - FS modulation

$$y(t) = x(t)e^{j4\omega_0 t}.$$

What are the new coefficients?

Answer

Here $m = 4$:

$$d_k = c_{k-4}.$$

The coefficient sequence shifts four harmonic slots.

Practice 11 - FS integration trap

Can a periodic signal with $c_0 = 3$ have a periodic antiderivative?

Answer

No. Nonzero average accumulates a ramp over time. A periodic antiderivative requires:

$$c_0 = 0.$$

Practice 12 - Find frequencies from a formula

$$x(t) = 3 \cos(8\pi t) - 2 \sin(20\pi t).$$

Answer

Compare to $\cos(2\pi ft)$ and $\sin(2\pi ft)$:

$$8\pi = 2\pi f \Rightarrow f = 4 \text{ Hz},$$

$$20\pi = 2\pi f \Rightarrow f = 10 \text{ Hz}.$$

CFT peaks appear at ± 4 and ± 10 Hz.

Practice 13 - 2D shift

An image is shifted 0.2 units right and 0.1 units up in the continuous coordinate system.

What happens to its 2D CFT magnitude?

Answer

Spatial shift only multiplies by a unit-magnitude phase factor:

$$G(u, v) = F(u, v)e^{-j2\pi(0.2u+0.1v)}.$$

Therefore:

$$|G(u, v)| = |F(u, v)|.$$

Practice 14 - 2D derivative

What frequency-domain operation approximates the horizontal derivative of an image?

Answer

$$\frac{\partial I}{\partial x} \leftrightarrow j2\pi u F(u, v).$$

The factor grows with $|u|$, so it emphasizes high horizontal spatial frequencies and therefore vertical edges.

Practice 15 - Low-pass vs high-pass

Why does a low-pass filtered image blur?

Answer

Sharp edges need high spatial frequencies. Removing high frequencies leaves only slowly varying spatial content, so transitions become smooth/blurred.

Practice 16 - Convolution theorem

An LTI system has impulse response $h(t)$. You know $X(f)$ and $H(f)$. What is the output spectrum?

Answer

$$Y(f) = X(f)H(f).$$

No convolution in frequency is needed; multiplication in frequency corresponds to convolution in time.

Practice 17 - Combined tricky transform

$$y(t) = 2x(3(t-1))e^{j2\pi 4t}.$$

Find $Y(f)$.

Answer

Start with:

$$g(t) = x(3(t-1)).$$

Scaling + shift:

$$G(f) = \frac{1}{3}X(f/3)e^{-j2\pi f}.$$

Then modulation by 4 Hz shifts $G(f)$:

$$Y(f) = 2G(f-4).$$

Therefore:

$$Y(f) = \frac{2}{3}X\left(\frac{f-4}{3}\right)e^{-j2\pi(f-4)}.$$

Practice 18 - Phase bug

Measured phase is 3.14 and predicted phase is -3.14 . Direct squared error is huge. Is the property necessarily wrong?

Answer

No. They are almost the same angle modulo 2π . Use wrapped phase difference:

```
diff = np.angle(np.exp(1j*(measured-predicted)))
```

30. Harder mini-lab practice problems

These are closer to the style you said may be harder/trickier than historical questions.

Hard Problem A - Shift + reverse + derivative

Given:

$$x(t) = e^{-t^2} \cos(2\pi 3t),$$

construct:

$$y(t) = \frac{d}{dt} x(-(t-1)).$$

Tasks:

1. Generate $x(t)$ and $y(t)$.
2. Numerically compute $X(f)$ and $Y(f)$.
3. Predict $Y(f)$ only from properties.
4. Compare magnitude and phase.

Answer/property derivation

Let:

$$g(t) = x(-t).$$

Then:

$$G(f) = X(-f).$$

Now $x(-(t-1)) = g(t-1)$:

$$H(f) = X(-f)e^{-j2\pi f}.$$

Differentiate:

$$Y(f) = j2\pi f X(-f)e^{-j2\pi f}.$$

Numerically get $X(-f)$ by complex interpolation.

Hard Problem B - Recover a filter's impulse response

Suppose an LTI system input/output are:

$$x(t), \quad y(t).$$

You numerically calculate $X(f)$ and $Y(f)$.

Where $X(f)$ is safely nonzero:

$$H(f) = \frac{Y(f)}{X(f)}.$$

Then inverse CFT:

$$h(t) = \mathcal{F}^{-1}\{H(f)\}.$$

Numerical caution

Never divide by near-zero values:

```
H = np.zeros_like(X, dtype=complex)

mask = np.abs(X) > 1e-3*np.max(np.abs(X))

H[mask] = Y[mask] / X[mask]
```

Then inverse-transform `H`.

This connects your earlier LTI/impulse-response material to CFT.

Hard Problem C - Detect and remove a sinusoidal noise tone

A measured signal is:

$$y(t) = x(t) + 0.7 \cos(2\pi f_n t).$$

Unknown f_n .

Tasks:

1. calculate $Y(f)$,
2. locate suspicious narrow symmetric peaks,
3. estimate f_n ,
4. zero/suppress a small band around $\pm f_n$,
5. inverse-transform,
6. compare original/cleaned waveform.

Key idea

A real cosine noise tone creates symmetric peaks at $\pm f_n$.

Hard Problem D - 2D notch filtering

An image has diagonal periodic stripe noise.

Tasks:

1. compute 2D CFT,
2. plot log magnitude,
3. locate off-center bright symmetric peaks,
4. notch them,
5. reconstruct.

Key idea

Do **not** use a high-pass filter blindly. Periodic noise may lie at specific mid/high frequencies. Remove only the suspicious symmetric peaks.

Hard Problem E - Compare time-domain derivative vs frequency-domain derivative

1. Generate a smooth finite-duration signal.
2. Calculate derivative with `np.gradient`.
3. CFT the derivative.
4. Predict derivative spectrum with $j2\pi fX$.
5. Alternatively calculate $j2\pi fX$ and inverse-transform it.
6. Compare both derivative waveforms.

Expected result

They should approximately agree away from finite-window boundary effects.

31. Debugging checklist for CFT questions

If a transform looks wrong, inspect in this order.

31.1 Wrong 2π convention

If your kernel is:

```
np.exp(-1j * 2*np.pi * f * t)
```

then sinusoid `sin(2*pi*5*t)` has frequency **5 Hz**.

But `sin(5*t)` has angular frequency 5 rad/s, which means:

$$f = 5/(2\pi) \text{ Hz.}$$

31.2 Frequency axis too narrow

A 15 Hz component cannot appear if:

```
f = np.linspace(-10, 10, 1000)
```

31.3 Time window too short

Truncating a Gaussian or other noncompact signal causes transform error.

31.4 Too few samples

Increase time and frequency samples.

31.5 Wrong shift direction

Right shift:

$$x(t - t_0).$$

Left shift:

$$x(t + t_0).$$

31.6 Time compression backwards

$$x(at), a > 1$$

is compressed, not expanded.

31.7 Modulation sign wrong

$$x(t)e^{+j2\pi f_0 t} \leftrightarrow X(f - f_0)$$

shifts **right**.

31.8 Phase MSE huge but magnitude perfect

Likely:

- phase wrapping,
 - phase compared where magnitude is near zero.
-

31.9 `np.trapezoid` wrong axis

Print shapes before/after:

```
print(A.shape)

B = np.trapezoid(A, axis_values, axis=...)

print(B.shape)
```

The integration axis must have the same length as `axis_values`.

31.10 `np.outer` orientation wrong

Remember:

```
np.outer(f, t).shape == (len(f), len(t))
```

Rows = first argument, columns = second.

32. Debugging checklist for Fourier Series

Wrong period

Use one complete period:

```
T = t[-1] - t[0]  
  
omega0 = 2*np.pi/T
```

Forgot negative harmonics

Use:

```
range(-N, N+1)
```

Forgot to calculate coefficients

```
fs.calculate_all_coefficients()
```

before `approximate()`.

Wrong exponent sign

Analysis:

$$e^{-jk\omega_0 t}$$

Synthesis:

$$e^{+jk\omega_0 t}.$$

Reconstruction poor

Possible causes:

- too few harmonics,
 - discontinuity/Gibbs phenomenon,
 - incorrect period,
 - incomplete sample period.
-

33. What to say about Gibbs phenomenon if asked

For a periodic signal with a jump discontinuity, a finite Fourier-Series reconstruction produces oscillatory overshoot near the jump.

Increasing the number of harmonics:

- makes the oscillation region narrower,
- does not make the maximum overshoot disappear completely in the simple partial-sum sense.

So if your square-wave FS reconstruction rings near edges, your code may still be correct.

34. Magnitude and phase: physical meaning

For any complex spectral value:

$$X(f) = |X(f)|e^{j\phi(f)}.$$

- $|X(f)|$: strength of that frequency component.
- $\phi(f)$: phase/alignment information.

For FS:

$$c_k = |c_k|e^{j\phi_k}.$$

- $|c_k|$: epicycle/vector radius or harmonic strength,
- ϕ_k : initial angle/phase.

Time shifts often leave magnitude unchanged while changing phase. This is why phase cannot be ignored when reconstructing a signal.

35. Short derivations worth being able to reproduce

CFT time shift

$$y(t) = x(t - t_0)$$
$$Y(f) = \int x(t - t_0) e^{-j2\pi ft} dt.$$

Let $\tau = t - t_0$, so $t = \tau + t_0$:

$$Y(f) = \int x(\tau) e^{-j2\pi f(\tau + t_0)} d\tau$$
$$= e^{-j2\pi ft_0} X(f).$$

CFT differentiation

$$Y(f) = \int x'(t) e^{-j2\pi ft} dt.$$

Integration by parts gives, for signals with suitable boundary behavior:

$$\boxed{Y(f) = j2\pi f X(f)}.$$

CFT time scaling

$$Y(f) = \int x(at) e^{-j2\pi ft} dt.$$

Let $\tau = at$, so $dt = d\tau/a$; accounting for reversal if $a < 0$ gives:

$$\boxed{Y(f) = \frac{1}{|a|} X(f/a)}.$$

FS time shift

$$y(t) = \sum_k c_k e^{jk\omega_0(t-t_0)}$$

$$= \sum_k (c_k e^{-jk\omega_0 t_0}) e^{jk\omega_0 t}.$$

Therefore:

$$d_k = c_k e^{-jk\omega_0 t_0}.$$

FS differentiation

$$\frac{d}{dt} (c_k e^{jk\omega_0 t}) = jk\omega_0 c_k e^{jk\omega_0 t}.$$

Therefore:

$$d_k = jk\omega_0 c_k.$$

36. Exam-ready templates

Template A - Generate -> CFT -> plot magnitude/phase

```
import numpy as np

import matplotlib.pyplot as plt

# axes

t = np.linspace(-5, 5, 2000)

f = np.linspace(-10, 10, 1000)

# signal

x = ...
```

```
# CFT

X = CFT1D(t, x, f).transform()

# magnitude

plt.plot(f, np.abs(X))

plt.grid(True)

plt.show()

# phase

plt.plot(f, np.angle(X))

plt.grid(True)

plt.show()
```

Template B - Verify a property

```
x = ...

y = ...

X = CFT1D(t, x, f).transform()

Y = CFT1D(t, y, f).transform()

Y_pred = ... # theoretical property
```

```
print("Complex MSE:", np.mean(np.abs(Y-Y_pred)**2))

print("Magnitude MSE:", np.mean((np.abs(Y)-np.abs(Y_pred))**2))
```

Template C - Parseval

```
X = CFT1D(t, x, f).transform()

Et = np.trapezoid(np.abs(x)**2, t)

Ef = np.trapezoid(np.abs(X)**2, f)

print("Time:", Et)

print("Freq:", Ef)

print("Error:", abs(Et-Ef))
```

Template D - Detect dominant frequencies

```
mag = np.abs(X)

mask = f > 0

fp = f[mask]

mp = mag[mask]
```

```

peaks = np.where(

    (mp[1:-1] > mp[:-2]) &

    (mp[1:-1] > mp[2:]))

)[0] + 1

top = peaks[np.argsort(mp[peaks])[-3:]]

print(np.sort(fp[top]))

```

Template E - FS analysis/reconstruction

```

fs = FourierEpicyles(t, x, n_harmonics=N)

fs.calculate_all_coefficients()


ns = np.array(sorted(fs.coeffs))

cs = np.array([fs.coeffs[k] for k in ns])


plt.stem(ns, np.abs(cs))

plt.show()


x_hat = fs.approximate(t)

```

Template F - 2D CFT spectrum + phase

```
img = ContinuousImage(path)

cft = CFT2D(img)

real, imag = cft.compute_cft()

mag = np.sqrt(real**2 + imag**2)

phase = np.arctan2(imag, real)

plt.imshow(np.log1p(mag), cmap='gray')

plt.colorbar()

plt.show()

plt.imshow(phase, cmap='twilight')

plt.colorbar()

plt.show()
```

37. Things you should NOT do unless explicitly allowed

1. Do not use `np.fft` when the question forbids built-in FT/DFT/FFT.
2. Do not manually hard-code a known Fourier transform if the task is to numerically calculate it.
3. Do not implement a continuous time shift using `np.roll` unless the problem is specifically discrete/circular shifting.
4. Do not compare phase blindly where magnitude is zero.
5. Do not forget negative FS harmonics.
6. Do not use `x/y` axes as `u/v` frequency axes in the 2D assignment.
7. Do not build a direct 4-deep (x, y, u, v) numerical 2D CFT if separability is required.

8. Do not return a complex image from the current inverse 2D CFT when the specification asks for the real reconstructed spatial signal.
-

38. Five-minute memory sheet

Fourier Series

$$\omega_0 = 2\pi/T$$

$$c_k = \frac{1}{T} \int_T x(t) e^{-jk\omega_0 t} dt$$

$$x(t) = \sum_k c_k e^{jk\omega_0 t}$$

Shift:

$$x(t - t_0) \leftrightarrow c_k e^{-jk\omega_0 t_0}$$

Reverse:

$$x(-t) \leftrightarrow c_{-k}$$

Differentiate:

$$x^{(n)}(t) \leftrightarrow (jk\omega_0)^n c_k$$

Modulate:

$$x(t) e^{jm\omega_0 t} \leftrightarrow c_{k-m}$$

Parseval:

$$\frac{1}{T} \int_T |x|^2 dt = \sum_k |c_k|^2$$

CFT

$$X(f) = \int x(t) e^{-j2\pi ft} dt$$

$$x(t) = \int X(f) e^{j2\pi ft} df$$

Shift:

$$x(t - t_0) \leftrightarrow X(f) e^{-j2\pi f t_0}$$

Reverse:

$$x(-t) \leftrightarrow X(-f)$$

Scale:

$$x(at) \leftrightarrow \frac{1}{|a|} X(f/a)$$

Modulate:

$$x(t) e^{j2\pi f_0 t} \leftrightarrow X(f - f_0)$$

Cosine modulation:

$$x(t) \cos 2\pi f_0 t \leftrightarrow \frac{1}{2} [X(f - f_0) + X(f + f_0)]$$

Differentiate:

$$x^{(n)}(t) \leftrightarrow (j2\pi f)^n X(f)$$

Convolution:

$$x * h \leftrightarrow XH$$

Parseval:

$$\int |x|^2 dt = \int |X|^2 df$$

Real signal:

$$X(-f) = X^*(f)$$

39. Final night-before checklist

You are ready if you can do all of these without looking up syntax:

- ☐ Generate a signal with `np.sin`, `np.cos`, `np.exp`.
 - ☐ Implement a piecewise signal with Boolean masks.
 - ☐ Shift, reverse, compress, expand, scale, and modulate a function.
 - ☐ Write a 1D numerical CFT using `np.outer` + `np.trapezoid`.
 - ☐ Write a numerical inverse CFT.
 - ☐ Plot waveform, magnitude, and phase.
 - ☐ Calculate magnitude/phase of complex values.
 - ☐ Calculate MSE and Parseval energy.
 - ☐ Explain time shift, reversal, scaling, modulation, differentiation, convolution, Parseval.
 - ☐ Apply those properties to both FS and CFT.
 - ☐ Extract FS coefficient arrays from your dictionary.
 - ☐ Explain negative FS harmonics and conjugate symmetry.
 - ☐ Understand 2D shapes `(Y,X)`, `(U,X)`, `(Y,U,X)`, `(V,Y,U)`, `(V,U)`.
 - ☐ Explain low-pass, high-pass, band-pass, and notch filtering.
 - ☐ Explain why edges are high frequency.
 - ☐ Recognize phase wrapping as a source of fake phase MSE.
 - ☐ Recognize finite-window/sampling error as a source of nonzero numerical MSE.
-

40. The highest-value things to practice tonight

If time is limited, do these in this exact order:

1. **Write the 1D CFT template from memory.**
2. **Time shift Gaussian:** verify magnitude + phase + complex MSE.
3. **Time scaling + modulation:** verify $\frac{1}{|a|}X((f - f_0)/a)$.
4. **Derivative:** verify first/second/third derivative using $(j2\pi f)^n X$.
5. **Parseval:** piecewise signal, compare energy integrals.
6. **FS:** shift one periodic signal and verify $c_k e^{-jk\omega_0 t_0}$.
7. **FS:** derivative property $(jk\omega_0)c_k$.
8. **2D CFT:** from memory explain every array shape in the forward and inverse methods.
9. **Image filtering:** explain what low/high/notch filters remove and why.
10. **Phase:** practice wrapped phase error and magnitude masking once.

If you can do those ten cleanly, you cover almost every pattern represented by the supplied historical questions and your current FS/CFT assignments.